

Car Maintenance Tracker & Inventory Management System

By Dane Shuler (Group Name: Car Guy)

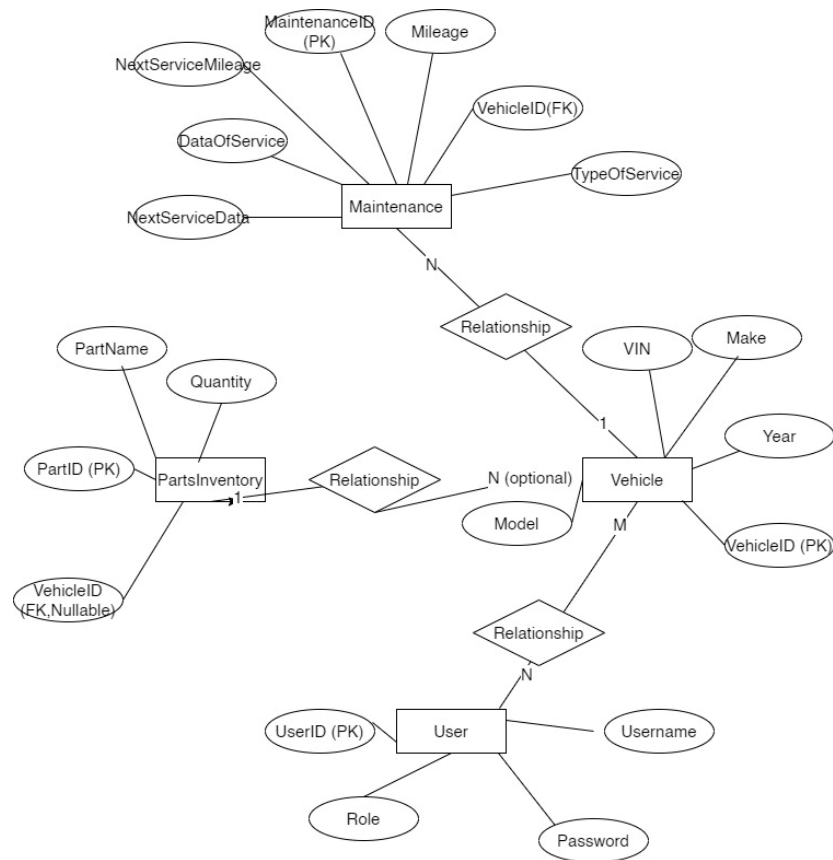
Problem Statement/Proposal:

The goal of this project is to create a command-line application designed to help car owners and small repair shops manage their vehicle maintenance schedules and parts inventory efficiently. This tool will allow users to log and track the maintenance history of their vehicles, manage an inventory of parts, and set reminders for upcoming maintenance tasks.

The application will be simple yet functional, with a focus on providing an easy-to-use interface within a command-line environment. Users will be able to input vehicle details, record maintenance activities, manage a parts inventory, and search for information based on various criteria.

The project will be completed within the semester, and its scope is designed to be realistic for a single developer. While the core functionality will focus on maintenance and inventory management, the structure will be built in a way that allows for future expansion.

Conceptual Database Design



Entities:

1. Vehicle

- Attributes:
 - VehicleID (Primary Key)
 - Make
 - Model
 - Year
 - VIN

2. Maintenance

- Attributes:
 - MaintenanceID (Primary Key)
 - VehicleID (Foreign Key)
 - DateOfService
 - TypeOfService
 - Mileage
 - NextServiceDate
 - NextServiceMileage

3. PartsInventory

- Attributes:
 - PartID (Primary Key)
 - PartName
 - Quantity
 - VehicleID (Foreign Key, Nullable)

4. User

- Attributes:
 - UserID (Primary Key)
 - Username
 - Password
 - Role (Admin/User)

Relationships:

1. Vehicle and Maintenance:

- One-to-Many (A vehicle can have multiple maintenance records, but a maintenance record is associated with only one vehicle).

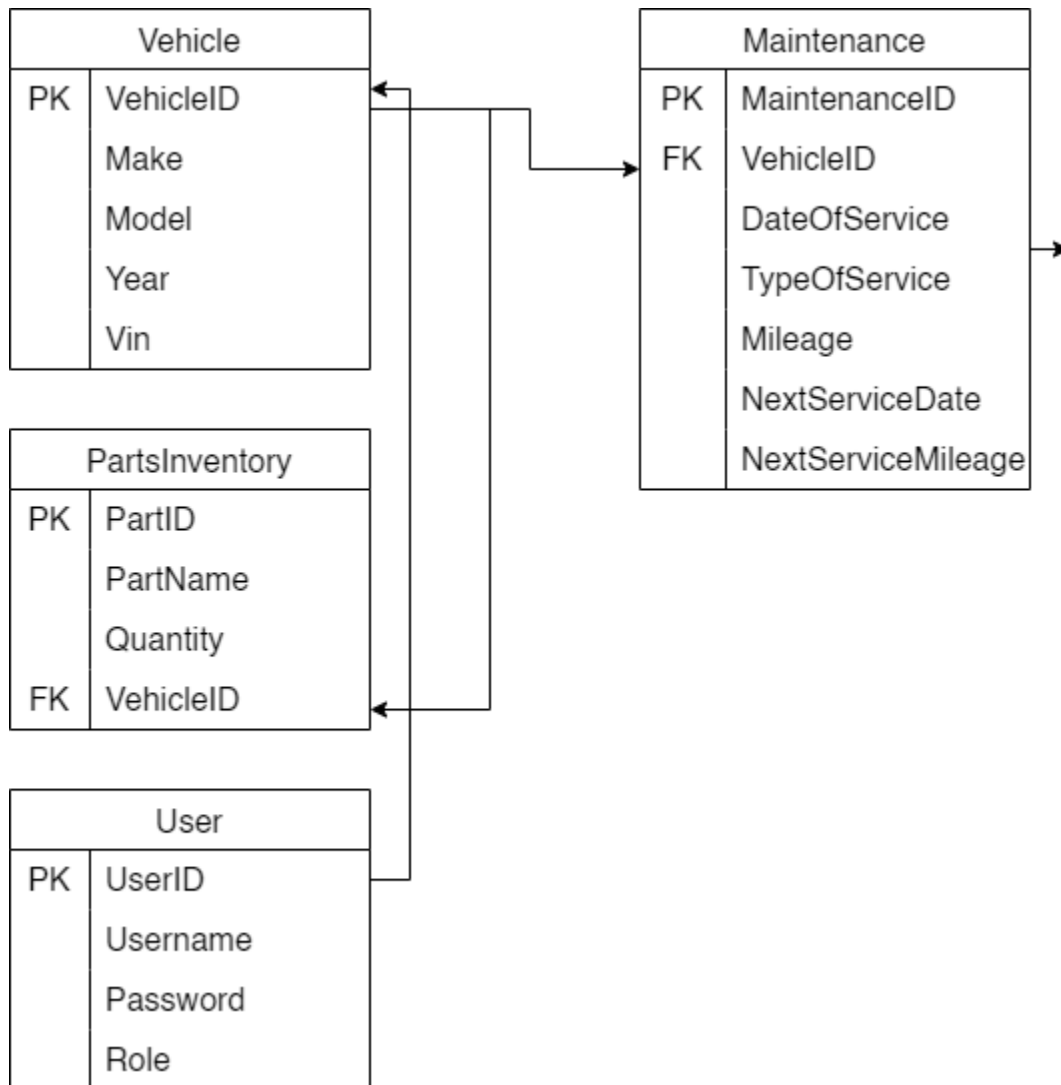
2. PartsInventory and Vehicle:

- Optional One-to-Many (A part may belong to a specific vehicle, but not necessarily. A vehicle can use multiple parts).

Assumptions:

- Each vehicle will have multiple maintenance records over time.
- Parts in the inventory can be associated with specific vehicles, but some parts may not belong to a specific vehicle.
- Users of the system will either have admin rights (to manage vehicles, maintenance records, and inventory) or regular user rights (to log and view records).

Logical Database Design



Summary Table of Data Types:

Table	Attribute	Type	Constraint
Vehicle	VehicleID	INT	Primary Key
	Make	VARCHAR(50)	NOT NULL
	Model	VARCHAR(50)	NOT NULL
	Year	INT	NOT NULL
	VIN	VARCHAR(17)	NOT NULL, UNIQUE
Maintenance	MaintenanceID	INT	Primary Key
	VehicleID	INT	Foreign Key references Vehicle.VehicleID
	DateOfService	DATE	NOT NULL
	TypeOfService	VARCHAR(100)	NOT NULL
	Mileage	INT	NOT NULL
	NextServiceDate	DATE	NULLABLE
	NextServiceMileage	INT	NULLABLE
PartsInventory	PartID	INT	Primary Key
	PartName	VARCHAR(100)	NOT NULL
	Quantity	INT	DEFAULT 0
	VehicleID	INT	Foreign Key references Vehicle.VehicleID (nullable)
User	UserID	INT	Primary Key
	Username	VARCHAR(50)	NOT NULL, UNIQUE
	Password	VARCHAR(50)	NOT NULL
	Role	ENUM('Admin', 'User')	NOT NULL

Functional Requirements:

Add Cars:

- Users can add vehicle details such as make, model, year, and VIN.

Maintenance Tracker:

- Log details of maintenance activities (date of service, type of service, mileage).
- Set reminders for the next maintenance based on mileage or time intervals.

Parts Inventory Management:

- Add, update, and track parts in inventory (name, quantity, vehicle association).
- Update inventory when parts are used or restocked.

Search and Filter:

- Search for vehicles, maintenance records, or parts by car model, service type, or part name.
- Filter maintenance records by date or mileage.

Edit and Delete Records:

- Modify or delete vehicle details, maintenance logs, and parts entries.

Reporting:

- Generate a vehicle's maintenance history or an inventory summary.

Application Program Design:

-- Function to add a new vehicle

-- Input: Make, Model, Year, VIN

INSERT INTO Vehicle (Make, Model, Year, VIN)

VALUES ('Toyota', 'Corolla', 2018, '123456789ABCDEFGH');

-- Check if VIN already exists before this insertion for error handling

-- Function to add a new maintenance record

-- Input: VehicleID, DateOfService, TypeOfService, Mileage, NextServiceDate, NextServiceMileage (optional)

INSERT INTO Maintenance (VehicleID, DateOfService, TypeOfService, Mileage, NextServiceDate, NextServiceMileage)

VALUES (123, '2023-10-17', 'Oil Change', 5000, '2024-01-17', 8000);

-- Ensure the VehicleID exists before adding the maintenance record

-- Function to update an existing maintenance record

-- Input: MaintenanceID, DateOfService, TypeOfService, Mileage, NextServiceDate, NextServiceMileage (optional)

UPDATE Maintenance

SET DateOfService = '2023-10-17', TypeOfService = 'Brake Replacement', Mileage = 6000

WHERE MaintenanceID = 456;

-- Use the MaintenanceID to locate and update the maintenance record

-- Function to delete a vehicle and associated maintenance records

-- Input: VehicleID

DELETE FROM Vehicle WHERE VehicleID = 123;

-- Cascades to delete related maintenance records for this VehicleID

-- Function to add a part to the inventory

-- Input: PartName, Quantity, VehicleID (optional)

INSERT INTO PartsInventory (PartName, Quantity, VehicleID)

VALUES ('Air Filter', 10, 123);

-- If VehicleID is provided, associate the part with the vehicle

-- Function to update part quantity in the inventory

-- Input: PartID, Quantity

UPDATE PartsInventory

SET Quantity = 15

WHERE PartID = 789;

-- Locate the part by PartID and update its quantity

-- Function to generate a maintenance history for a vehicle

-- Input: VehicleID

```
SELECT * FROM Maintenance
```

```
WHERE VehicleID = 123
```

```
ORDER BY DateOfService DESC;
```

-- Retrieve and sort all maintenance records by date for the given VehicleID

-- Function to search for vehicles by make and model

-- Input: Make, Model

```
SELECT * FROM Vehicle
```

```
WHERE Make = 'Toyota' AND Model = 'Corolla';
```

-- Display all vehicles matching the specified make and model

-- Function to list all parts in the inventory

```
SELECT PartName, Quantity FROM PartsInventory;
```

-- Retrieve all parts and their quantities from the inventory

-- Aggregation: Get total parts used from inventory

```
SELECT SUM(InitialQuantity - Quantity) AS TotalPartsUsed
```

```
FROM PartsInventory;
```

-- Calculate the total number of parts used based on initial and current quantities

-- Aggregation: Get the maintenance record with the highest mileage

```
SELECT * FROM Maintenance
```

```
ORDER BY Mileage DESC
```

```
LIMIT 1;
```

-- Find and display the maintenance record with the highest mileage

-- Advanced query: Get vehicles due for maintenance based on mileage or service date

```
SELECT * FROM Maintenance
```

```
WHERE NextServiceDate <= CURDATE() OR NextServiceMileage <= 500;
```

```
-- Retrieve vehicles that need maintenance soon (based on date or mileage)
```

Installation Instructions for Car Maintenance Tracker

Intended Operating System

- Linux (Preferred: Ubuntu/Debian-based distributions)
- Windows 10/11 (with Python and MySQL pre-installed)

Steps to Install the System on Another Computer

1. Clone or Copy the Project Files

- Download or clone the repository containing the following files:
 - setup.sh (Installation script)
 - car_maintenance_web.py (Flask application)
 - setup_database.sql (Database schema and sample data)
- If you receive a zipped file, extract the contents to your desired directory.

2. Run the Installation Script

- **For Linux:**
 1. Open a terminal and navigate to the project directory:
 2. `cd /path/to/project`
 3. Make the setup script executable:
 4. `chmod +x setup.sh`
 5. Run the installation script:
 6. `./setup.sh`
- **For Windows:**
 1. Install Python, MySQL, and pip manually as outlined in the dependencies section below.
 2. Execute the setup_database.sql file in MySQL Workbench to set up the database.

3. Start the Application

- Once the installation is complete, start the Flask application:
- `python3 car_maintenance_web.py`
- The application will start running on <http://127.0.0.1:5000>.

4. Access the Application

- Open a web browser and navigate to <http://127.0.0.1:5000>.
- Use the user-friendly interface to manage vehicles, parts, and maintenance records.

Dependencies

1. **Programming Language:**
 - Python 3.x
2. **Database:**
 - MySQL (Tested with MySQL 8.0)
3. **Python Libraries:**
 - Flask (pip install flask)
 - MySQL Connector (pip install mysql-connector-python)
4. **Additional Tools** (Linux only):
 - Bash for running the setup script.
5. **System Requirements:**
 - RAM: Minimum 2 GB (Recommended: 4 GB)
 - Disk Space: Minimum 500 MB

User Manual for Car Maintenance Tracker Website

This guide will walk you through the steps for using the Car Maintenance Tracker Website.

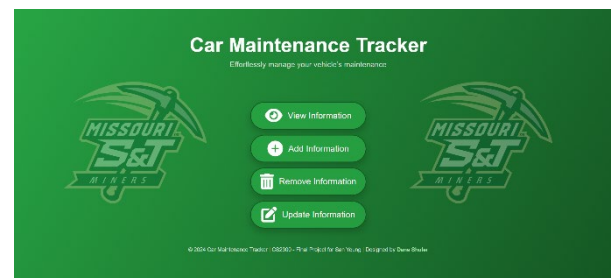
Accessing the Website

1. Open a web browser on your computer.
2. Navigate to the website URL: <http://127.0.0.1:5000> (if hosted locally).

Main Menu

The main menu contains the following options:

1. **View:** View vehicles, parts, and maintenance records.
2. **Add:** Add new vehicles, parts, and maintenance records.
3. **Remove:** Delete existing vehicles, parts, and maintenance records.
4. **Update:** Update details of existing vehicles, parts, and maintenance records.

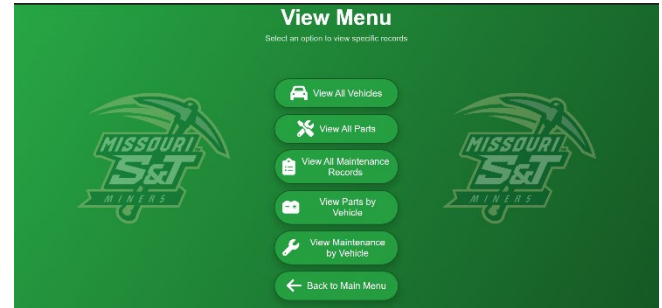


Clicking any option will take you to the corresponding submenu.

Step-by-Step Guide for Each Feature

1. Viewing Records

- **How to view records:**
 1. From the main menu, click the **"View"** option.
 2. Select one of the following options:
 - View All Vehicles
 - View All Parts
 - View All Maintenance Records
 - View Parts by Vehicle
 - View Maintenance Records by Vehicle
 3. A table or list of records will be displayed for the selected category.
- **Example:**
 - Click **"View All Vehicles"** to see a list of all vehicles in the system.



View Vehicles

Below are the details for all Vehicles records.

VehicleID	Make	Model	Year	VIN
1	Toyota	Corolla	2018	JTDBU4EE9A1234567
2	Chevy	Corvette	2008	1G1YY25W285124928
3	Honda	Civic	2020	2HGFC2F6XH1234568
4	Ford	F-150	2015	1FTFW1EF1FKD84332
5	BMW	X5	2021	5UXCR6C09M9H12345
6	Tesla	Model S	2022	5YJSA1E22JF123456
7	Toyota	Camry	2019	4T1B11HK6KU123456
8	Chevy	Silverado	2020	1GCPYBEH1LZ123456
9	Honda	Accord	2021	1HGCV1F37MA123456
10	Ford	Mustang	2017	1FA6P8CF6H5301234
11	BMW	3 Series	2020	WBA8D9C56KA123456
12	Tesla	Model 3	2022	5YJ3E1EB9JF123456

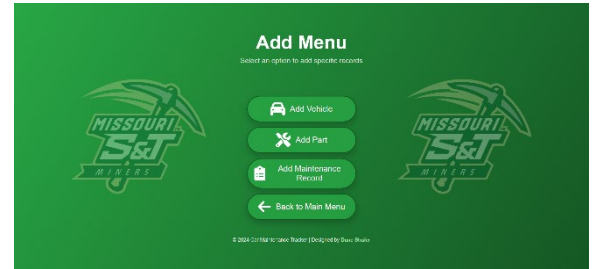
2. Adding Records

- **How to add a record:**

1. From the main menu, click the **"Add"** option.
2. Select one of the following:
 - Add Vehicle
 - Add Part
 - Add Maintenance Record
3. Fill out the form fields on the page.
 - Example for adding a vehicle:
 - Enter the vehicle's make, model, year, and VIN.
4. Click the **"Add"** button to save the record.
5. A confirmation message will appear upon success.

- **Example:**

- To add a part:
 1. Click **"Add Part"**.
 2. Fill in the part name, quantity, and (optional) associated vehicle ID.
 3. Click **"Add"** to save the part.



Add Part

Part Name:

Quantity:

Associated Vehicle:

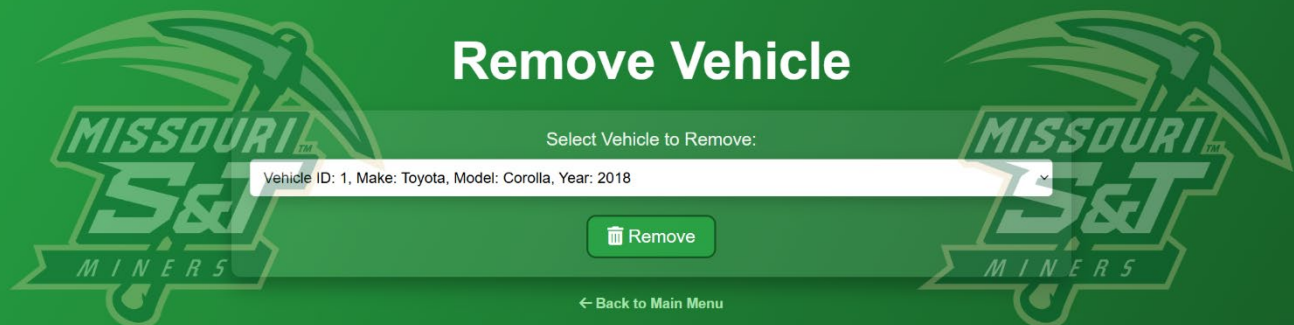
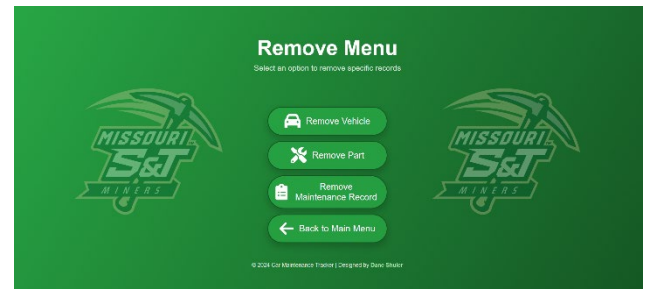
None

Add Part

[Back to Add Menu](#)

3. Removing Records

- **How to remove a record:**
 1. From the main menu, click the **"Remove"** option.
 2. Select one of the following:
 - Remove Vehicle
 - Remove Part
 - Remove Maintenance Record
 3. A dropdown menu will appear listing all records of the selected type.
 4. Select the record you want to delete and click **"Remove"**.
 5. A confirmation message will appear upon success.
- **Example:**
 - To remove a vehicle:
 1. Click **"Remove Vehicle"**.
 2. Select the vehicle from the dropdown menu.
 3. Click **"Remove"**.



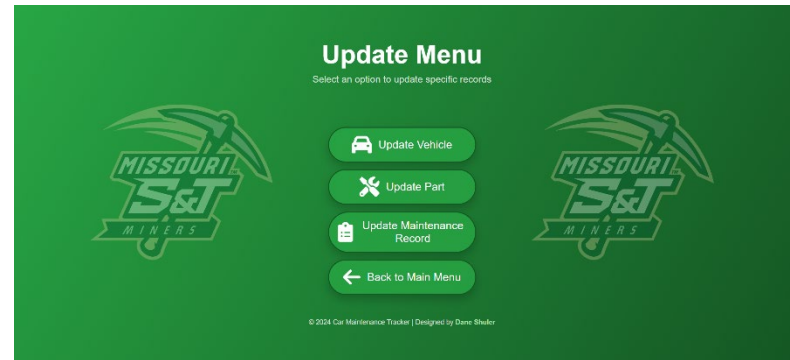
4. Updating Records

- **How to update a record:**

1. From the main menu, click the **"Update"** option.
2. Select one of the following:
 - Update Vehicle
 - Update Part
 - Update Maintenance Record
3. A dropdown menu will appear listing all records of the selected type.
4. Select the record you want to update and fill in the fields you wish to modify.
5. Click **"Update"** to save changes.
6. A confirmation message will appear upon success.

- **Example:**

- To update a maintenance record:
 1. Click **"Update Maintenance Record"**.
 2. Select the record from the dropdown menu.
 3. Modify the service date or type of service as needed.
 4. Click **"Update"** to save changes.



Update Maintenance

Navigation Tips

- Use the **Back to Main Menu** link on each page to return to the main menu.
- Error messages will be displayed if:
 - Required fields are left blank.
 - Invalid data is entered.
- Confirmation messages will indicate the success of an operation.

System Features Overview

1. **Responsive Web Interface:**
 - User-friendly interface to manage all records.
2. **Dynamic Dropdown Menus:**
 - Only relevant records are shown for removal and updates.
3. **Real-time Updates:**
 - All changes are immediately reflected in the database.

Sample Use Case

Add a Maintenance Record:

1. Navigate to **Add** → **Add Maintenance Record**.
2. Fill out the form:
 - Vehicle ID: 1
 - Date of Service: 2023-11-20
 - Type of Service: Oil Change
 - Mileage: 10,000
 - Next Service Date: 2024-03-20
 - Next Service Mileage: 15,000
3. Click **Add**.
4. Navigate to **View** → **View Maintenance Records** to verify the addition.

Troubleshooting

1. **Cannot Connect to Database:**
 - Ensure the MySQL server is running.
 - Verify the database credentials in the car_maintenance_web.py file.
2. **Page Not Found:**
 - Check the URL in the browser.
 - Verify that the Flask server is running.
3. **Dropdowns are Empty:**
 - Ensure the database contains records for the selected category.